Routing, Templating
& Static Files

Module 2: how Flask handles user requests, renders dynamic web layouts, and serves client assets.

Presented by Prof. Nilesh Panchotiya

How Does Flask Display a Webpage?

A user never communicates directly with Python. Web requests pass through an architectural pipeline before returning styled visual content.



Key Takeaway: Flask acts as a middleware bridge, transforming backend Python data structures into standardized HTML markup that web browsers understand.

What is URL Routing?

Routing Definition

Routing is the process of mapping an incoming HTTP client URL to a specific Python view function inside your Flask application.

Operational Flow

When a user requests : `/about`

```
URL Request [/about]
  → Flask checks Routing Table
  → Invokes function: about()
  → Sends Response to Client
```

```python
# basic_app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
  return "Welcome to Home Page"

@app.route("/about")
def about():
  return "About Us Page"
```

How Routing Works Internally

The Routing Request Cycle

- ✓ **1. HTTP Request:** Client sends a request, e.g., `GET /about`.
- ✓ **2. Werkzeug Map:** Flask uses Werkzeug's routing engine to inspect registered endpoints.
- ✓ **3. Decorator Match:** Finds matching view function decorated with `@app.route('/about')`.
- ✓ **4. View Execution:** Executes `about()` and obtains the return value.
- ✓ **5. HTTP Response:** Wraps data into an HTTP Response with status 200 OK.

Flask URL Map Inspection

Inside Python code, you can print the routing table:

```
print(app.url_map)

# Output Werkzeug Routing Table:
# Map([<Rule '/about' (GET) → about>,
#      <Rule '/' (GET) → home>])
```

Classification of Flask Routes

Static Routes

Fixed Path

Exact string endpoints that never change regardless of user input or parameters.

```
@app.route("/about")
@app.route("/contact")
@app.route("/login")
```

Dynamic Routes

Variable Path

Endpoints that accept dynamic parameter values directly from the requested URL path.

```
@app.route("/student/<name>")
@app.route("/product/<int:id>")
@app.route("/profile/<username>")
```

DYNAMIC URLS

Handling Dynamic Routes

Dynamic routes allow Flask to extract variables directly from the requested URL pattern using syntax.

```python
@app.route("/student/<name>")
def student(name):
    # 'name' is passed automatically
    return f"Hello {name}"
```

The parameter name specified inside must exactly match the view function argument name!

Execution Example

Client Request URL:
http://localhost:5000/student/Rahul

Rendered Browser Output:

```
Hello Rahul!
```

Flask URL Converters

Converter	Description & Example Path
`string`	Default. Accepts text without slashes (/user/Rahul)
`int`	Accepts positive integers (/product/10)
`float`	Accepts positive float values (/price/12.5)
`path`	Like string, but accepts slashes (/docs/python/flask)
`uuid`	Accepts UUID strings (/item/123e4567-e89b...)

```python
# Typed route converter implementation
@app.route("/age/<int:age>")
def age(age):
    # View must return a string/response
    return str(age)
```

Binding Multiple Routes

Flask allows stacking multiple `@app.route()` decorators over a single view function to bind multiple URLs.

```
@app.route("/")
@app.route("/home")
def home():
    return "Home Page"
```

Key Advantages

- **Flexible Navigation:** Enables intuitive aliases for key landing pages.
- **Backward Compatibility:** Keeps legacy URLs functioning during app refactoring.
- **DRY Principle:** Avoids duplicating identical view function logic across files.

The Danger of Hardcoding URLs

⚠ Hardcoded Approach (Anti-Pattern)

```
<!-- HTML file -->
<a href="/about">About</a>
```

Problem: If routing rules change from `/about` to `/company/about` in Python, every HTML file breaks and requires manual editing.

✓ Dynamic Solution: url_for()

```
<!-- Flask Jinja2 Template -->
<a href="{{ url_for('about') }}">About</a>
```

Solution: Dynamically look up endpoint URLs by function name instead of static strings.

Dynamic URL Generation with url_for()

Definition

`url_for()` is a Flask function that generates a URL path for a given target view function name dynamically.

Standard Syntax

```
url_for('function_name')
```

```
# Python View Function
@app.route("/about")
def about():
    return render_template("about.html")

<!-- HTML Template Link -->
<a href="{{ url_for('about') }}">
    About
</a>
```

Why Use url_for()?

Avoids Hard Coding

Change route URLs in Python without breaking templates or user navigation links.

Easy Maintenance

Centralized URL definitions drastically reduce bug hunting when paths change.

Supports Parameters

Pass dynamic arguments and query parameters seamlessly into generated paths.

url_for() with Parameters

```python
# Python View Function
@app.route("/user/<username>")
def user(username):
    return username
```

```html
<!-- HTML Template Link -->
<a href="{{ url_for('user', username='Rahul') }}">
    Profile
</a>
```

Generated Output URL

Flask automatically replaces the path variable:

```
/user/Rahul
```

Introduction to Templates

✕ Problem: Inline Strings

```python
@app.route("/")
def home():
    return "<h1>Hello</h1>"
```

Returning HTML strings inside Python files is not practical for building real applications.

✓ Solution: render_template()

`HTML File` → `render_template()` → `Browser`

Keeps presentation HTML files completely separate from backend Python server logic.

TEMPLATING

What is the Jinja2 Engine?

Jinja2 is Flask's built-in, fast, expressible, and secure dynamic templating engine.



HTML Structure
Standard web document layout



Python Data
Variables, dicts & lists



Logic Operations
If-else & loop constructs



Rendered Output
Pure HTML sent to client

Rendering Templates with render_template()

```python
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")
```

Directory Requirement

Flask looks inside a folder specifically named `templates` by default:

```
project/
├── app.py
└── templates/
    └── index.html
```

Passing Data to HTML Templates

1. Python Code (app.py)

```python
@app.route("/")
def home():
    return render_template(
        "index.html",
        name="Rahul",
        age=20
    )
```

2. Jinja2 Template (index.html)

```
<h1>{{ name }}</h1>
<p>{{ age }}</p>
```

Rendered Browser Output:

Rahul
20

Passing Lists to Templates

Python List

```python
@app.route("/students")
def student_list():
    students = ["Rahul", "Amit", "Neha"]
    return render_template(
        "students.html",
        students=students
    )
```

HTML / Jinja Loop

```
<ul>
{% for s in students %}
    <li>{{ s }}</li>
{% endfor %}
</ul>
```

Essential Jinja2 Syntax

{{ name }} Variables

Interpolates and prints variable values directly to HTML output.

```
<h1>{{ name }}</h1>
```

{% if ... %} Conditionals

Controls logical flow inside HTML templates.

```
{% if age > 18 %}
    Adult
{% endif %}
```

{% for ... %} Loops

Iterates over Python lists, tuples, or dictionaries.

```
{% for x in students %}
    {{ x }}
{% endfor %}
```

{# comment #} Comments

Server-side Jinja2 comments hidden from browser view source.

```
{# This is a Jinja2 comment #}
```

INHERITANCE

The Problem of Code Duplication

Without Template Inheritance

Every single HTML file must repeat common layout tags:

- Identical tags & CSS links
- Duplicated navigation bar headers
- Repeated footer copyright widgets

With Template Inheritance

Define global boilerplate once inside : `base.html`

- Child pages extend the base layout
- Only override specific dynamic content blocks
- 100% DRY (Don't Repeat Yourself) architecture

Architecture of base.html

Parent Layout (base.html)

```
<html>
<head><title>My Site</title></head>
<body>
    <header>Header & Nav</header>

    {% block content %}
    <!-- Child page content injected here -->
    {% endblock %}

    <footer>Footer</footer>
</body>
</html>
```

Extending Child Pages

Multiple templates inherit from base:

`home.html` `about.html` `contact.html`

`{% block content %}` defines an extendable region that child templates override.

Template Inheritance Example

Child Template (home.html)

```
{% extends "base.html" %}

{% block content %}

<h1>Home</h1>

{% endblock %}
```

Primary Advantages

- **Reusable Layouts:** Standardize app design across all screens.
- **Less Duplication:** Stop copying headers/footers into every file.
- **Easy Maintenance:** Edit layout once in `base.html` to update whole website.

TEMPLATE FILTERS

Transforming Data with Jinja2 Filters

Filters modify variable display values. Syntax: `{{ variable | filter_name }}`

Filter	Jinja2 Example Expression	Input Value	Output Value
`upper`	{{ name | upper }}	"rahul"	**RAHUL**
`lower`	{{ name | lower }}	"FLASK"	**flask**
`title`	{{ city | title }}	"new york"	**New York**
`length`	{{ students | length }}	['A', 'B', 'C']	**3**

Additional Template Filters

`default` Fallback

Provides default value if variable is missing:

```
{{ name | default("Guest") }}
```

`reverse` Sequence Inversion

Reverses string or list ordering:

```
{{ text | reverse }}
```

`join` String Concatenation

Joins list items with a separator string:

```
{{ students | join(", ") }}
```

`safe` Raw HTML Escape Bypass

Renders string without escaping HTML tags:

```
{{ html | safe }}
```

Managing Static Files

Static files never change dynamically per request and are served directly to web browsers.

Static Asset Types

- **CSS:** Stylesheets for visuals
- **JavaScript:** Client-side interactive scripts
- **Images:** Logos, photos, icons
- **Media & Fonts:** Custom typography, audio, video

Folder Directory Structure

```
project/
├── app.py
├── templates/
└── static/
    ├── css/
    ├── js/
    └── images/
```

Linking CSS Files

File Path

```
static/css/style.css
```

Always use `url_for('static', filename='...')` to ensure paths stay valid!

HTML Link Tag Syntax

```
<link
    rel="stylesheet"
    href="{{ url_for('static', filename='css/style.css') }}"
>
```

Including JavaScript Files

File Path

`static/js/script.js`

Script tags loaded dynamically resolve through Flask's static endpoint.

HTML Script Tag Syntax

```
<script src="{{ url_for('static', filename='js/script.js') }}"></script>
```

Displaying Image Assets

File Path

```
static/images/logo.png
```

HTML Image Tag Syntax

```
<img src="{{ url_for('static', filename='images/logo.png') }}">
```



Static Image Asset Rendered in Browser

PROJECT SETUP

Standard Project Structure

```
my_flask_project/
│
├── app.py          # Main Flask App & Routes
├── templates/      # Jinja2 HTML Templates
│   ├── base.html
│   ├── index.html
│   └── about.html
│
├── static/         # Public Static Assets
│   ├── css/style.css
│   ├── js/script.js
│   └── images/logo.png
│
└── venv/           # Python Virtual Environment
```

Key Directories

- **app.py:** Contains URL routes, view logic & app initialization.
- **templates/:** Dedicated folder for Jinja2 rendered HTML files.
- **static/:** Publicly accessible stylesheets, scripts & branding assets.

PROJECT FLOW

End-to-End Application Flow

1. GET /
Client Request

→

2. @app.route("/")
Route Match

→

3. home()
Python View

→

4. render_template()
Pass Data

→

5. index.html
Jinja2 Engine

Static Asset Resolution during Step 5:

When rendering , the browser requests , , and through the static route handler to display the final styled web page.